

FIT5137 Assignment 2 Report

MonCity dimensional warehouse and analytics

Semester 2 2026

Zekai Zheng

Student ID S35649682

Final report

15 September 2026

Operational data preparation, two dimensional schema designs, Oracle implementation, validation and Power BI analysis.

Contents

Main report	Pages 1 to 16
Appendix A Complete cleaning SQL	Pages A1 to A2

FIT5137 Assignment 2 Report

MonCity dimensional warehouse and analytics

Zekai Zheng
S35649682

Report component only

REPORT OVERVIEW

Executive summary

This report designs, deploys and validates a dimensional warehouse for the MonCity autonomous-vehicle database. The deployed Version 1 schema answers all seven prescribed questions at the highest safe aggregation level. Version 2 redesigns each subject at level 0 to preserve atomic events and future analytical flexibility.

The source contains 10,001 booking rows, 1,001 maintenance rows and 994 accident rows. Cleaning removes one duplicate booking, rejects one accident with no identifier, maps two invalid references to explicit UNKNOWN members, treats a negative maintenance amount as unknown, and corrects two error-message spellings. The MonCity source tables were not modified.

Recommended investment: run a controlled Mini Bus procurement pilot. Mini Bus demand (3,313 bookings) is close to the fleet maximum, while it has the lowest known maintenance cost per booking (\$26.74) and the fewest maintenance records (312). The decision remains conditional because one Mini Bus maintenance cost is unknown and the accident data has no date or exposure measure.

Safety priority: investigate image-recognition failure first. It produces the largest severe-damage count (51), even though low battery is the most frequent recorded error overall (369) and produces no severe-damage records in this dataset.

Controls passed: 10,000 cleaned bookings, 1,001 maintenance records and 993 valid accidents reconcile exactly to Version 1 fact totals; all group allocation weights sum to one; every A2 constraint is enabled and validated.

Deliverable map

Criterion	Evidence in this report
C2	Operational ERD, exploration SQL, cleaning rules, before/after evidence
C3	Version 1 and Version 2 dimensional designs for booking, maintenance and accident
C4	Oracle deployment, table structures, reconciliation and seven prescribed queries
C5	Two Power BI Web visuals, findings, recommendation and limitations

SOURCE UNDERSTANDING

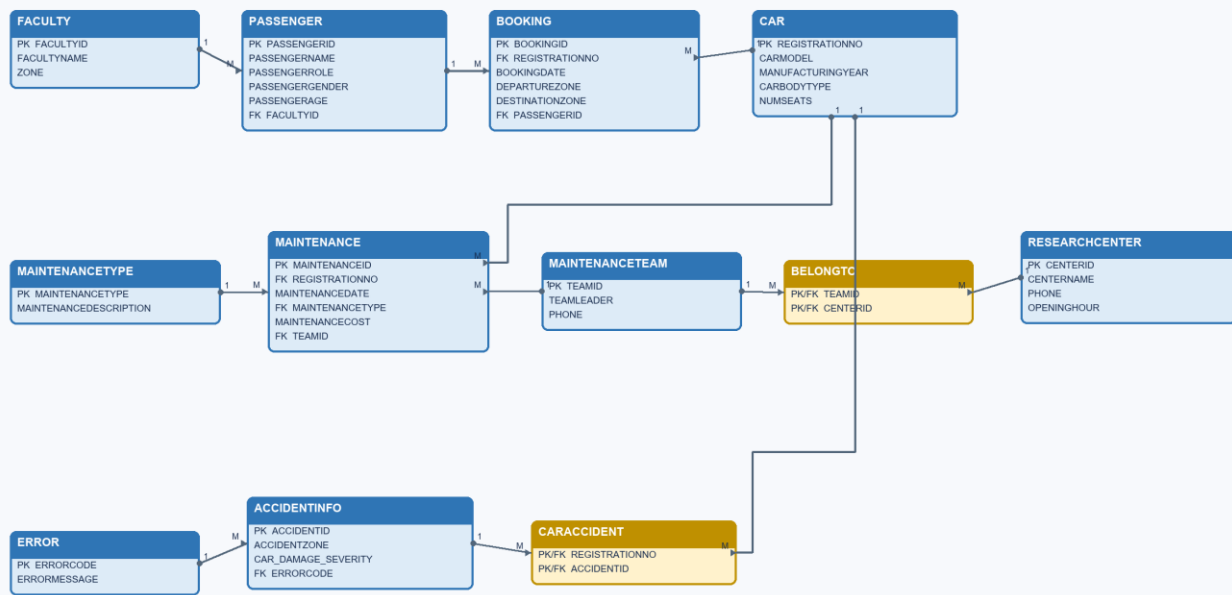
C2. Operational database model

The operational schema separates master data (car, passenger, faculty, maintenance type, team, centre and error) from transaction and relationship tables. Booking links a passenger and a car; maintenance links a car, team and maintenance type; accident information links to cars through CARACCIDENT. BELONGTO creates a many-to-many relationship between teams and research centres.

This relationship structure drives the dimensional solution: centre membership and multi-car accidents need determinant group dimensions plus bridges so that an event is stored once while membership remains queryable without double counting.

MonCity Operational Database ERD

Source schema reconstructed from the assignment specification



Relationship tables resolve CAR-ACCIDENT and TEAM-RESEARCH CENTRE many-to-many relationships.

Figure 1. Reconstructed MonCity operational ERD used for profiling and dimensional design.

Key grain observations

Operational subject	Transaction grain	Relationship implication
Booking	One booking identifier	Passenger and car each resolve to one row after cleaning
Maintenance	One maintenance identifier	A team can belong to multiple research centres
Accident	One accident identifier	An accident can involve multiple cars

READ-ONLY PROFILING

C2. SQL exploration and source profile

Exploration was performed in the S35649682 Oracle schema using read-only SELECT statements. The sequence was: inventory tables, compare row counts to distinct keys, test critical nulls, test orphan references, inspect numeric ranges and list exact duplicate business keys.

```

SELECT 'BOOKING' AS table_name, COUNT(*) AS row_count
FROM MONCITY.BOOKING
UNION ALL
SELECT 'MAINTENANCE', COUNT(*) FROM MONCITY.MAINTENANCE
UNION ALL
SELECT 'ACCIDENTINFO', COUNT(*) FROM MONCITY.ACCIDENTINFO;

```

```

SELECT bookingid, COUNT(*) AS copies
FROM MONCITY.BOOKING
GROUP BY bookingid
HAVING COUNT(*) > 1;

```

```

SELECT 'PASSENGER -> FACULTY' AS relationship, COUNT(*) AS orphan_count
FROM MONCITY.PASSENGER p
LEFT JOIN MONCITY.FACULTY f ON UPPER(TRIM(p.facultyid)) = UPPER(TRIM(f.facultyid))
WHERE f.facultyid IS NULL;

```

Source row counts

Table	Rows
ACCIDENTINFO	994
BELONGTO	12
BOOKING	10,001

Table	Rows
CAR	30
CARACCIDENT	1,000
ERROR	5
FACULTY	5
MAINTENANCE	1,001
MAINTENANCETEAM	5
MAINTENANCETYPE	5
PASSENGER	154
RESEARCHCENTER	4

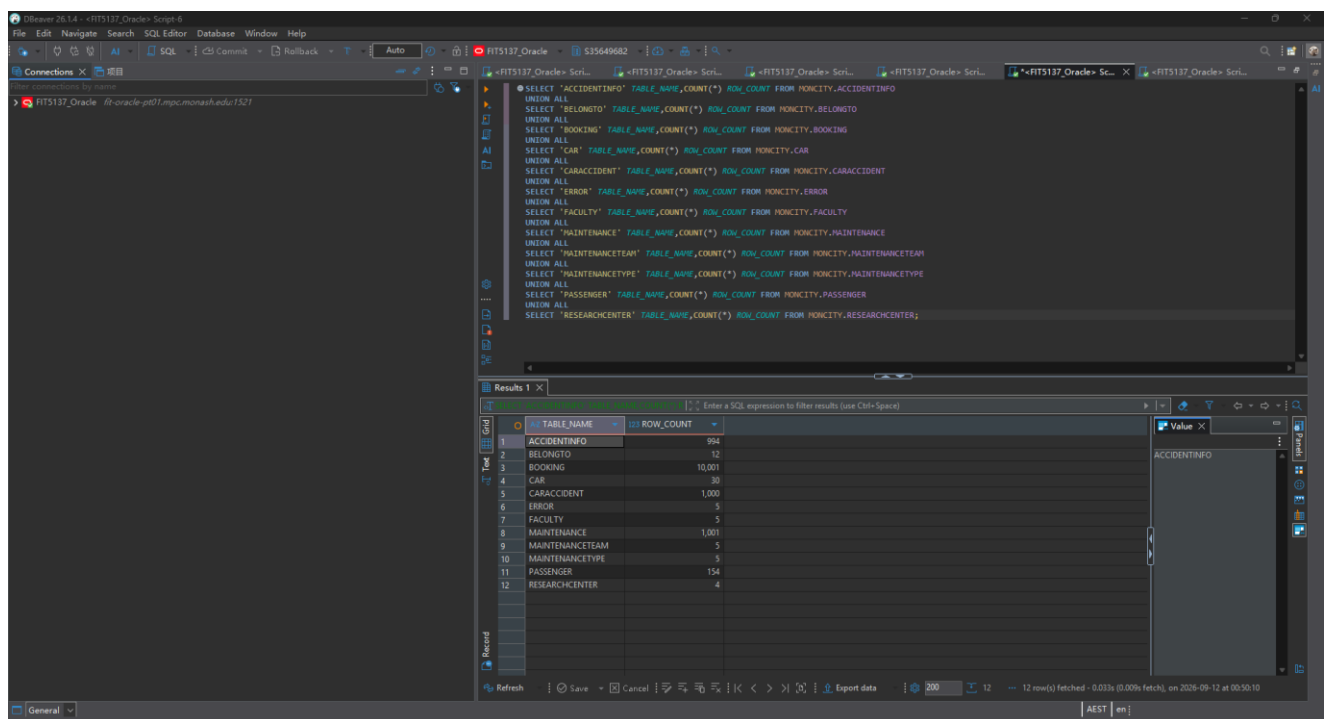


Figure 2. DBeaver execution evidence for the source inventory query.

CONTROLLED REMEDIATION

C2. Data-quality findings and cleaning

Cleaning is implemented in A2 staging views and conformed dimensions. It is deterministic, traceable and non-destructive: MONCITY remains the system of record. Unknown reference values are preserved through explicit UNKNOWN members, while records that cannot support a stable grain are rejected to a separate view.

Finding	Source evidence	Treatment
Duplicate booking	T1218 appears twice	SELECT DISTINCT produces one booking
Missing accident ID	1 row	Reject from fact; retain in A2_V_REJECT_ACCIDENT
Invalid faculty	U163 -> Alienware	Map to UNKNOWN faculty
Invalid error	A2000 -> Error010	Map to UNKNOWN error
Negative cost	M2000 -> -200	Set amount NULL and COST_UNKNOWN=1
Spelling errors	systeem; Unknow issue	Correct in error dimension
Age boundary	Age 60 exists	Old-aged range is 60-120

Core cleaning SQL

Complete cleaning and dimension statements are provided in Appendix A, pages A1-A2.

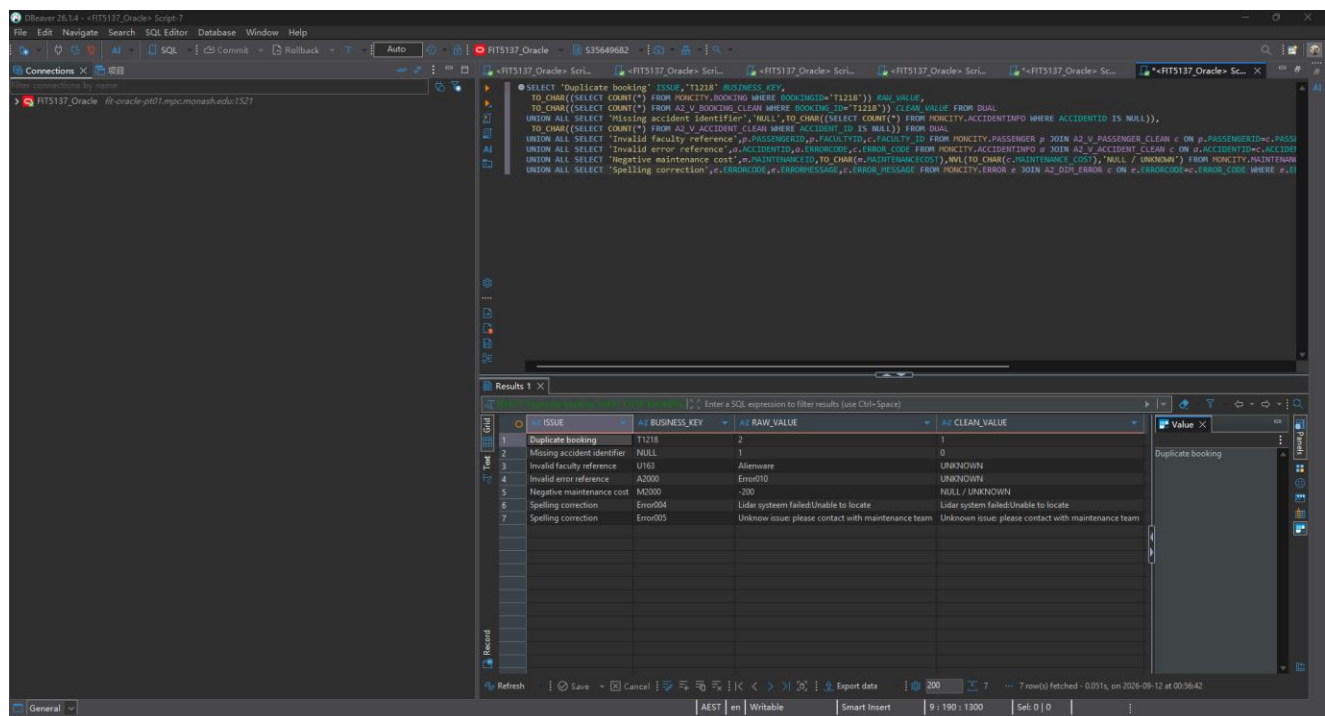
```
CREATE VIEW A2_V_PASSENGER_CLEAN AS
SELECT p.PASSENGERID, p.PASSENGERAGE,
       NVL(f.FACULTYID, 'UNKNOWN') AS FACULTY_ID
FROM MONCITY.PASSENGER p LEFT JOIN MONCITY.FACULTY f
  ON UPPER(TRIM(p.FACULTYID))=UPPER(TRIM(f.FACULTYID));

CREATE VIEW A2_V_BOOKING_CLEAN AS
SELECT DISTINCT b.BOOKINGID AS BOOKING_ID, b.BOOKINGDATE AS BOOKING_DATE,
       EXTRACT(MONTH FROM b.BOOKINGDATE) AS MONTH_ID,
       NVL(p.FACULTY_ID, 'UNKNOWN') AS FACULTY_ID,
       CASE WHEN p.PASSENGERAGE BETWEEN 18 AND 35 THEN 1
            WHEN p.PASSENGERAGE BETWEEN 36 AND 59 THEN 2
            WHEN p.PASSENGERAGE BETWEEN 60 AND 120 THEN 3 ELSE 0 END AS AGE_GROUP_ID,
       UPPER(TRIM(b.REGISTRATIONNO)) AS REGISTRATION_NO
FROM MONCITY.BOOKING b LEFT JOIN A2_V_PASSENGER_CLEAN p
  ON b.PASSENGERID=p.PASSENGERID
WHERE b.BOOKINGID IS NOT NULL;
```

VERIFIED ORACLE OUTPUT

C2. Before-and-after cleaning evidence

The following DBeaver result grid compares each profiled issue to the transformed value actually present in the A2 views or dimensions. It demonstrates row-level lineage rather than relying only on aggregate totals.



Issue	A2 BUSINESS_KEY	A2 RAW_VALUE	A2 CLEAN_VALUE
Duplicate booking	T1218	2	1
Missing accident identifier	NULL	1	0
Invalid faculty reference	U163	Alienware	UNKNOWN
Invalid error reference	A5000	Error010	UNKNOWN
Negative maintenance cost	M2000	-20	NULL / UNKNOWN
Spelling correction	Error004	Lidar system failedUnable to locate	Lidar system failedUnable to locate
Spelling correction	Error005	Unknown issue: please contact with maintenance team	Unknown issue: please contact with maintenance team

Figure 3. Oracle before/after evidence for all material cleaning rules.

HIGHEST SAFE AGGREGATION

C3. Dimensional schema Version 1

Version 1 stores only the dimensions and measures needed to answer the prescribed analytical questions. Each subject uses a separate fact table because booking, maintenance and accident events have different grains and different relationship paths.

Fact	Declared grain	Additive measures	Reason for aggregation
A2_FACT_BOOKING	Month x faculty x age group x body type	Number of bookings	Answers July, faculty, age and body-type questions without individual event detail
A2_FACT_MAINT	Body type x maintenance type x centre group	Record count, known cost, unknown-cost count	Supports type and centre analysis while keeping each event once
A2_FACT_ACCIDENT	Car group x zone x error x severity	Number of accidents	Supports zone, car, error and severity questions without duplicating multi-car events

Bridge design

A centre-group determinant represents each distinct set of centres attached to a maintenance team. A car-group determinant represents each distinct set of cars attached to an accident. Each determinant stores weight factor 1/n and connects to a bridge. The fact references the group once; EXISTS is used for membership questions and weights are available for allocated totals.

Month-of-year is intentionally used for Version 1 because the required question is 'July' and the source horizon does not require cross-year analysis. Version 2 restores the full booking date.

Version 1 contains 17 implemented tables: 10 regular dimensions, two group dimensions, two bridges and three fact tables.

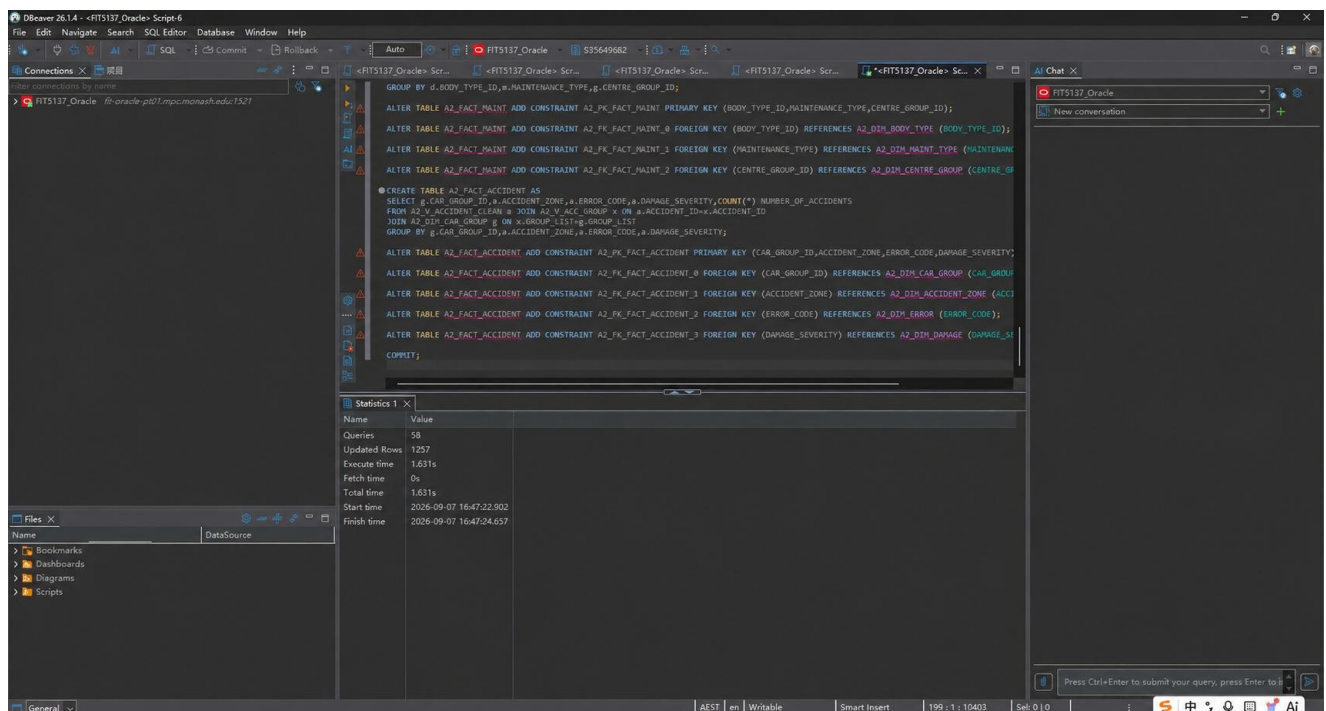


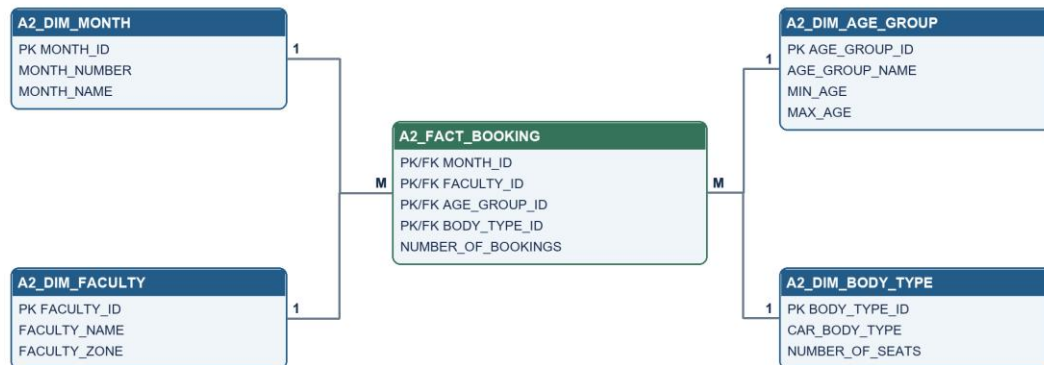
Figure 4. DBeaver execution statistics: 58 statements completed successfully and 1,257 rows inserted/updated.

DIMENSIONAL DESIGN

C3. Version 1 - booking subject

Version 1 | Booking subject

Highest required aggregation; only Version 1 is implemented.



1:M = parent-to-child cardinality | Dashed boundary = determinant/group | DD = degenerate dimension

Figure 5. Version 1 booking star schema at month/faculty/age-group/body-type grain.

The fact's composite primary key enforces one row for every dimensional combination.

NUMBER_OF_BOOKINGS is fully additive over all four dimensions. Faculty and age group are resolved through the cleaned passenger view; body type is resolved through the car dimension.

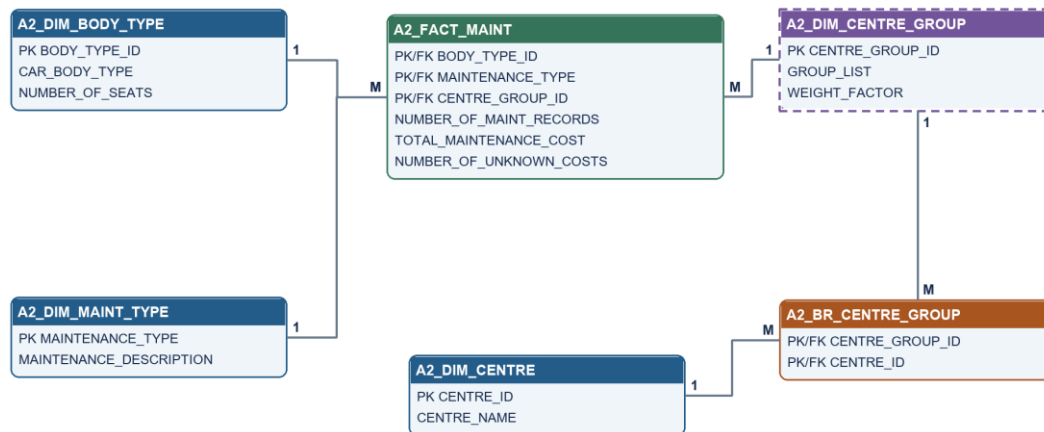
Age groups: Young adult 18-35; Middle-aged adult 36-59; Old-aged adult 60-120; Other/unknown for values outside the controlled ranges.

DIMENSIONAL DESIGN

C3. Version 1 - maintenance subject

Version 1 | Maintenance subject

Highest required aggregation; only Version 1 is implemented.



1:M = parent-to-child cardinality | Dashed boundary = determinant/group | DD = degenerate dimension

Figure 6. Version 1 maintenance star with centre-group determinant and bridge.

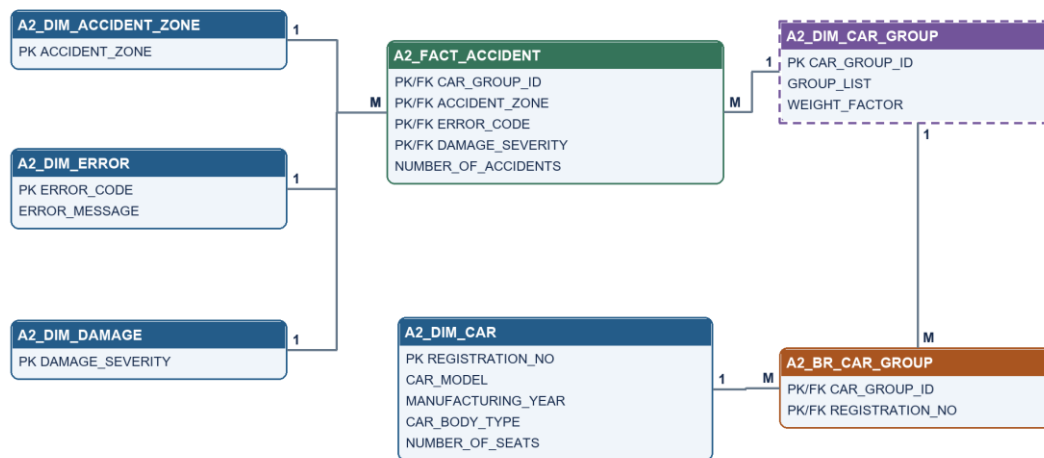
TOTAL_MAINTENANCE_COST sums only valid non-negative amounts. NUMBER_OF_UNKNOWN_COSTS preserves the count of records whose amount cannot be used. Centre membership is queried through A2_BR_CENTRE_GROUP so that CE04 analysis includes all relevant team-centre memberships without multiplying the maintenance fact.

DIMENSIONAL DESIGN

C3. Version 1 - accident subject

Version 1 | Accident subject

Highest required aggregation; only Version 1 is implemented.



1:M = parent-to-child cardinality | Dashed boundary = determinant/group | DD = degenerate dimension

Figure 7. Version 1 accident star with car-group determinant and bridge.

NUMBER_OF_ACCIDENTS is stored once per accident group, zone, error and severity. Car membership is resolved through A2_BR_CAR_GROUP. This design correctly handles eight multi-car accidents and an accident with no car link (mapped to UNKNOWN).

LEVEL 0 REDESIGN

C3. Dimensional schema Version 2

Version 2 redesigns all three subjects at atomic level 0. User-created roll-up dimensions in Version 1 are removed where the natural operational detail is available. Atomic identifiers and dates are retained, allowing new analyses without rebuilding facts.

Atomic fact	Level 0 grain	Restored detail	Removed Version 1 roll-up
FACT_BOOKING_L0	One cleaned booking	Booking ID, booking date, passenger, car	Month and age-group-only grain
FACT_MAINT_L0	One maintenance event	Maintenance ID/date, car, team, amount	Body-type and centre-group aggregate grain
FACT_ACCIDENT_L0	One valid accident	Accident ID with car-group membership	Zone/error/severity aggregate grain

The accident bridge is still structurally necessary because one atomic accident can involve multiple cars. For maintenance, the source team is retained at level 0 and centre membership can be resolved through the original team-centre relationship rather than pre-aggregating to centre groups.

Trade-off

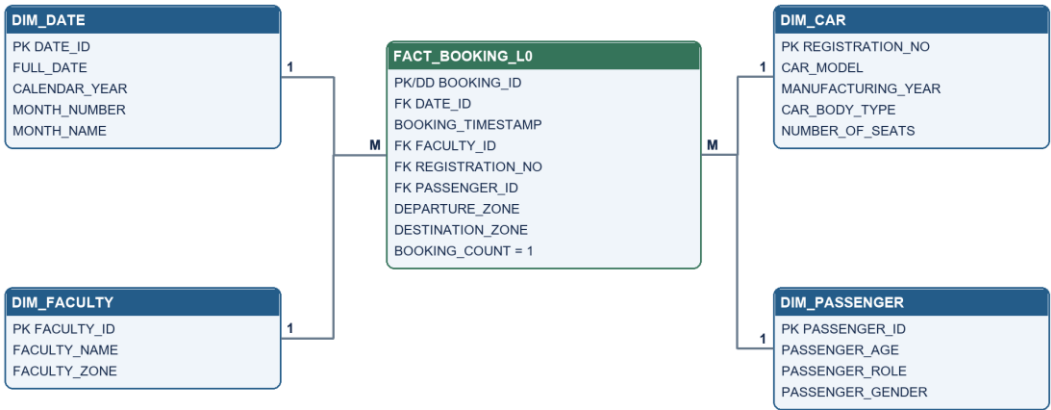
Version 1 is compact and optimised for the given questions; Version 2 is more flexible, auditable and suitable for date-sensitive analysis, but stores more rows and pushes some grouping work to queries or semantic-model measures.

ALTERNATIVE DESIGN

C3. Version 2 - booking at level 0

Version 2 | Booking subject

Level 0 design only: one row per identified event; no aggregation.



1:M = parent-to-child cardinality | Dashed boundary = determinant/group | DD = degenerate dimension

Figure 8. Atomic booking fact with individual booking, date, passenger and car dimensions.

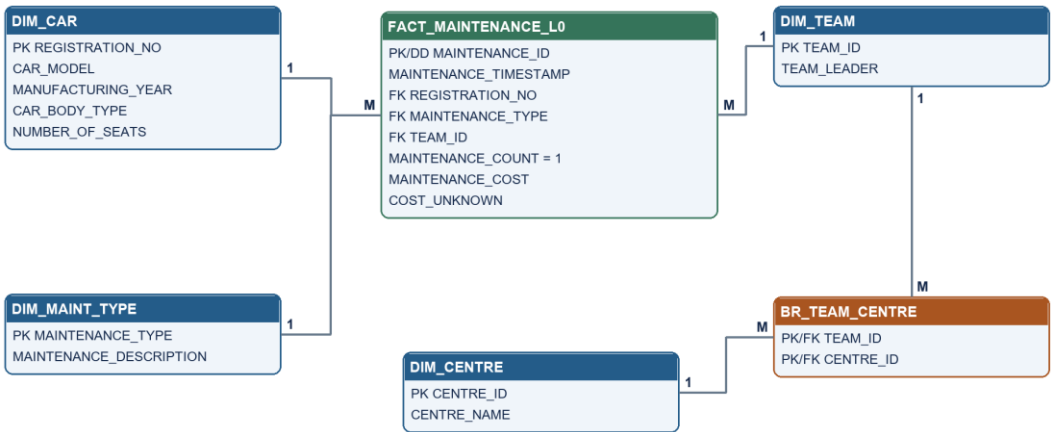
Age and faculty remain attributes of the passenger and faculty path; analysts can define alternative age bands without reloading the fact.

ALTERNATIVE DESIGN

C3. Version 2 - maintenance at level 0

Version 2 | Maintenance subject

Level 0 design only: one row per identified event; no aggregation.



1:M = parent-to-child cardinality | Dashed boundary = determinant/group | DD = degenerate dimension

Figure 9. Atomic maintenance fact with event amount and original team membership.

The individual maintenance amount remains nullable and the unknown-cost flag remains explicit, preserving both financial integrity and record completeness.

ALTERNATIVE DESIGN

C3. Version 2 - accident at level 0

Version 2 | Accident subject

Level 0 design only: one row per identified event; no aggregation.

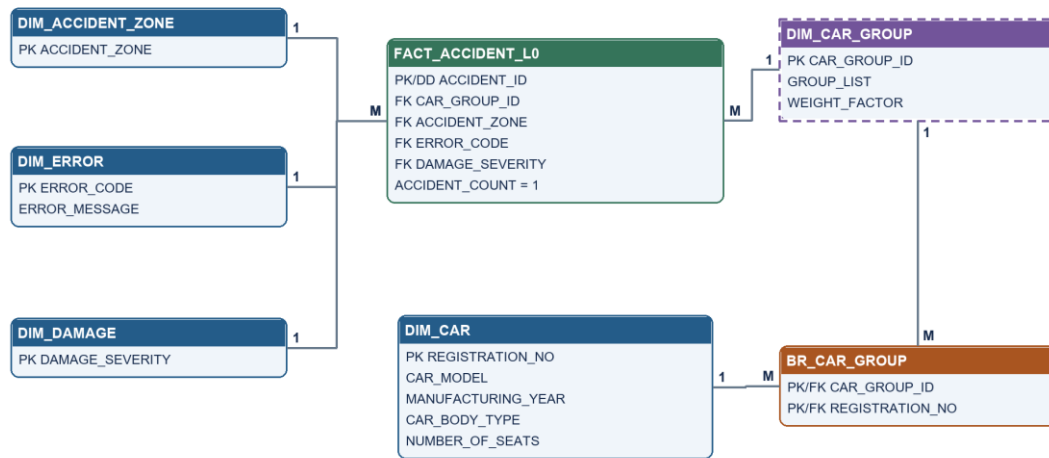


Figure 10. Atomic accident fact with determinant car group and bridge.

The accident identifier becomes the fact key; zone, error and severity remain directly analysable while car membership remains many-to-many.

VERSION 1 DEPLOYMENT

C4. Oracle implementation

The Version 1 model was created in schema S35649682 with primary keys, foreign keys and validated bridge structures. Views isolate cleaning rules; dimensions are loaded first; bridges are loaded next; fact tables are aggregated last.

```

CREATE TABLE A2_FACT_BOOKING AS
SELECT b.month_id, b.faculty_id, b.age_group_id, d.body_type_id,
       COUNT(*) AS number_of_bookings
FROM A2_V_BOOKING_CLEAN b
JOIN A2_DIM_CAR c ON b.registration_no = c.registration_no
JOIN A2_DIM_BODY_TYPE d
  ON c.car_body_type = d.car_body_type
 AND c.number_of_seats = d.number_of_seats
GROUP BY b.month_id, b.faculty_id, b.age_group_id, d.body_type_id;
    
```

```

ALTER TABLE A2_FACT_BOOKING ADD CONSTRAINT A2_PK_FACT_BOOKING
PRIMARY KEY (month_id, faculty_id, age_group_id, body_type_id);
    
```

```

CREATE VIEW A2_V_ACC_GROUP AS
SELECT accident_id,
       LISTAGG(registration_no, ',') WITHIN GROUP (ORDER BY registration_no) AS group_list,
       1 / COUNT(*) AS weight_factor
FROM (SELECT DISTINCT accident_id, registration_no FROM A2_V_ACC_CAR)
GROUP BY accident_id;
    
```

Implemented object inventory

Object	Rows
A2_DIM_MONTH	12
A2_DIM_FACULTY	6
A2_DIM_AGE_GROUP	4
A2_DIM_BODY_TYPE	3
A2_DIM_CAR	31
A2_DIM_MAINT_TYPE	5
A2_DIM_CENTRE	4

Object	Rows
A2_DIM_CENTRE_GROUP	4
A2_BR_CENTRE_GROUP	8
A2_DIM_ACCIDENT_ZONE	4
A2_DIM_ERROR	6
A2_DIM_DAMAGE	5
A2_DIM_CAR_GROUP	39
A2_BR_CAR_GROUP	47
A2_FACT_BOOKING	468
A2_FACT_MAINT	60
A2_FACT_ACCIDENT	551

All deployed objects use the A2_ prefix so they are isolated from the MONCITY operational source. No DROP or source UPDATE statements are used in the verified deployment script.

ORACLE METADATA EVIDENCE

C4. Implemented structures - booking

	AZ TABLE_NAME	123 POS	AZ COLUMN_NAME	AZ DATA_TYPE	123 BYTES	123 PREC	123 SCALE	AZ NULLABLE
1	A2_DIM_AGE_GROUP	1	AGE_GROUP_ID	NUMBER	22	[NULL]	[NULL]	N
2	A2_DIM_AGE_GROUP	2	AGE_GROUP_NAME	VARCHAR2	17	[NULL]	[NULL]	Y
3	A2_DIM_AGE_GROUP	3	MIN_AGE	NUMBER	22	[NULL]	[NULL]	Y
4	A2_DIM_AGE_GROUP	4	MAX_AGE	NUMBER	22	[NULL]	[NULL]	Y
5	A2_DIM_BODY_TYPE	1	BODY_TYPE_ID	NUMBER	22	[NULL]	[NULL]	N
6	A2_DIM_BODY_TYPE	2	CAR_BODY_TYPE	VARCHAR2	26	[NULL]	[NULL]	Y
7	A2_DIM_BODY_TYPE	3	NUMBER_OF_SEATS	NUMBER	22	38	0	Y
8	A2_DIM_FACULTY	1	FACULTY_ID	VARCHAR2	26	[NULL]	[NULL]	N
9	A2_DIM_FACULTY	2	FACULTY_NAME	VARCHAR2	256	[NULL]	[NULL]	Y
10	A2_DIM_FACULTY	3	FACULTY_ZONE	VARCHAR2	128	[NULL]	[NULL]	Y
11	A2_DIM_MONTH	1	MONTH_ID	NUMBER	22	[NULL]	[NULL]	N
12	A2_DIM_MONTH	2	MONTH_NUMBER	NUMBER	22	[NULL]	[NULL]	Y
13	A2_DIM_MONTH	3	MONTH_NAME	VARCHAR2	36	[NULL]	[NULL]	Y
14	A2_FACT_BOOKING	1	MONTH_ID	NUMBER	22	[NULL]	[NULL]	N
15	A2_FACT_BOOKING	2	FACULTY_ID	VARCHAR2	26	[NULL]	[NULL]	N
16	A2_FACT_BOOKING	3	AGE_GROUP_ID	NUMBER	22	[NULL]	[NULL]	N
17	A2_FACT_BOOKING	4	BODY_TYPE_ID	NUMBER	22	[NULL]	[NULL]	N
18	A2_FACT_BOOKING	5	NUMBER_OF_BOOKINGS	NUMBER	22	[NULL]	[NULL]	Y

Figure 11. USER_TAB_COLUMNS evidence for booking dimensions and fact.

18 metadata rows are shown in full. POS is column order; BYTES is storage length; PREC and SCALE are numeric precision and scale. NULLABLE indicates whether a null is permitted.

ORACLE METADATA EVIDENCE

C4. Implemented structures - maintenance

user_tab_columns 1 X									
Enter a SQL expression to filter results (use Ctrl+Space)									
	AZ TABLE_NAME	123 POS	AZ COLUMN_NAME	AZ DATA_TYPE	123 BYTES	123 PREC	123 SCALE	AZ NULLABLE	
1	A2_BR_CENTRE_GROUP	1	CENTRE_GROUP_ID	NUMBER	22	[NULL]	[NULL]	N	
2	A2_BR_CENTRE_GROUP	2	CENTRE_ID	VARCHAR2	26	[NULL]	[NULL]	N	
3	A2_DIM_BODY_TYPE	1	BODY_TYPE_ID	NUMBER	22	[NULL]	[NULL]	N	
4	A2_DIM_BODY_TYPE	2	CAR_BODY_TYPE	VARCHAR2	26	[NULL]	[NULL]	Y	
5	A2_DIM_BODY_TYPE	3	NUMBER_OF_SEATS	NUMBER	22	38	0	Y	
6	A2_DIM_CENTRE	1	CENTRE_ID	VARCHAR2	26	[NULL]	[NULL]	N	
7	A2_DIM_CENTRE	2	CENTRE_NAME	VARCHAR2	128	[NULL]	[NULL]	Y	
8	A2_DIM_CENTRE_GROUP	1	CENTRE_GROUP_ID	NUMBER	22	[NULL]	[NULL]	N	
9	A2_DIM_CENTRE_GROUP	2	GROUP_LIST	VARCHAR2	4,000	[NULL]	[NULL]	Y	
10	A2_DIM_CENTRE_GROUP	3	WEIGHT_FACTOR	NUMBER	22	[NULL]	[NULL]	Y	
11	A2_DIM_MAINT_TYPE	1	MAINTENANCE_TYPE	VARCHAR2	26	[NULL]	[NULL]	N	
12	A2_DIM_MAINT_TYPE	2	MAINTENANCE_DESCRIPTION	VARCHAR2	128	[NULL]	[NULL]	Y	
13	A2_FACT_MAINT	1	BODY_TYPE_ID	NUMBER	22	[NULL]	[NULL]	N	
14	A2_FACT_MAINT	2	MAINTENANCE_TYPE	VARCHAR2	26	[NULL]	[NULL]	N	
15	A2_FACT_MAINT	3	CENTRE_GROUP_ID	NUMBER	22	[NULL]	[NULL]	N	
16	A2_FACT_MAINT	4	NUMBER_OF_MAINT_RECORDS	NUMBER	22	[NULL]	[NULL]	Y	
17	A2_FACT_MAINT	5	TOTAL_MAINTENANCE_COST	NUMBER	22	[NULL]	[NULL]	Y	
18	A2_FACT_MAINT	6	NUMBER_OF_UNKNOWN_COSTS	NUMBER	22	[NULL]	[NULL]	Y	

Figure 12. USER_TAB_COLUMNS evidence for maintenance dimensions, bridge and fact.

18 metadata rows are shown in full. POS is column order; BYTES is storage length; PREC and SCALE are numeric precision and scale. NULLABLE indicates whether a null is permitted.

ORACLE METADATA EVIDENCE

C4. Implemented structures - accident

user_tab_columns 1 X									
Enter a SQL expression to filter results (use Ctrl+Space)									
	AZ TABLE_NAME	123 POS	AZ COLUMN_NAME	AZ DATA_TYPE	123 BYTES	123 PREC	123 SCALE	AZ NULLABLE	
1	A2_BR_CAR_GROUP	1	CAR_GROUP_ID	NUMBER	22	[NULL]	[NULL]	N	
2	A2_BR_CAR_GROUP	2	REGISTRATION_NO	VARCHAR2	26	[NULL]	[NULL]	N	
3	A2_DIM_ACCIDENT_ZONE	1	ACCIDENT_ZONE	VARCHAR2	26	[NULL]	[NULL]	N	
4	A2_DIM_CAR	1	REGISTRATION_NO	VARCHAR2	26	[NULL]	[NULL]	N	
5	A2_DIM_CAR	2	CAR_MODEL	VARCHAR2	26	[NULL]	[NULL]	Y	
6	A2_DIM_CAR	3	MANUFACTURING_YEAR	NUMBER	22	[NULL]	[NULL]	Y	
7	A2_DIM_CAR	4	CAR_BODY_TYPE	VARCHAR2	26	[NULL]	[NULL]	Y	
8	A2_DIM_CAR	5	NUMBER_OF_SEATS	NUMBER	22	[NULL]	[NULL]	Y	
9	A2_DIM_CAR_GROUP	1	CAR_GROUP_ID	NUMBER	22	[NULL]	[NULL]	N	
10	A2_DIM_CAR_GROUP	2	GROUP_LIST	VARCHAR2	4,000	[NULL]	[NULL]	Y	
11	A2_DIM_CAR_GROUP	3	WEIGHT_FACTOR	NUMBER	22	[NULL]	[NULL]	Y	
12	A2_DIM_DAMAGE	1	DAMAGE_SEVERITY	VARCHAR2	26	[NULL]	[NULL]	N	
13	A2_DIM_ERROR	1	ERROR_CODE	VARCHAR2	26	[NULL]	[NULL]	N	
14	A2_DIM_ERROR	2	ERROR_MESSAGE	VARCHAR2	4,000	[NULL]	[NULL]	Y	
15	A2_FACT_ACCIDENT	1	CAR_GROUP_ID	NUMBER	22	[NULL]	[NULL]	N	
16	A2_FACT_ACCIDENT	2	ACCIDENT_ZONE	VARCHAR2	26	[NULL]	[NULL]	N	
17	A2_FACT_ACCIDENT	3	ERROR_CODE	VARCHAR2	26	[NULL]	[NULL]	N	
18	A2_FACT_ACCIDENT	4	DAMAGE_SEVERITY	VARCHAR2	26	[NULL]	[NULL]	N	
19	A2_FACT_ACCIDENT	5	NUMBER_OF_ACCIDENTS	NUMBER	22	[NULL]	[NULL]	Y	

Figure 13. USER_TAB_COLUMNS evidence for accident dimensions, bridge and fact.

19 metadata rows are shown in full. POS is column order; BYTES is storage length; PREC and SCALE are numeric precision and scale. NULLABLE indicates whether a null is permitted.

EVIDENCE OF CORRECTNESS

C4. Validation and reconciliation

Fact totals were reconciled to cleaned staging views and, where meaningful, to the raw source. The raw known-cost total of \$302,500 includes the invalid -\$200 value. After that amount is converted to unknown, the valid known-cost total becomes \$302,700 and reconciles exactly to the maintenance fact.

Measure	Raw	Clean	Warehouse	Interpretation
Bookings	10,001	10,000	10,000	Exact after cleaning
Maintenance records	1,001	1,001	1,001	Exact after cleaning
Known maintenance cost	302,500	302,700	302,700	Exact after cleaning
Accidents	994	993	993	Exact after cleaning

Structural controls

Control	Failures
Car groups with invalid weight sum	0
Centre groups with invalid weight sum	0
Disabled or unvalidated constraints	0

Validation SQL

```
SELECT 'Bookings' MEASURE, (SELECT COUNT(*) FROM MONCITY.BOOKING) RAW_VALUE,
(SELECT COUNT(*) FROM A2_V_BOOKING_CLEAN) CLEAN_VALUE, (SELECT SUM(NUMBER_OF_BOOKINGS) FROM
A2_FACT_BOOKING) WAREHOUSE_VALUE FROM DUAL
UNION ALL SELECT 'Maintenance records', (SELECT COUNT(*) FROM MONCITY.MAINTENANCE),
(SELECT COUNT(*) FROM A2_V_MAINT_CLEAN), (SELECT SUM(NUMBER_OF_MAINT_RECORDS) FROM A2_FACT_MAINT) FROM
DUAL
UNION ALL SELECT 'Known maintenance cost', (SELECT SUM(MAINTENANCECOST) FROM MONCITY.MAINTENANCE),
(SELECT SUM(MAINTENANCE_COST) FROM A2_V_MAINT_CLEAN), (SELECT SUM(TOTAL_MAINTENANCE_COST) FROM
A2_FACT_MAINT) FROM DUAL
UNION ALL SELECT 'Accidents', (SELECT COUNT(*) FROM MONCITY.ACCIDENTINFO),
(SELECT COUNT(*) FROM A2_V_ACCIDENT_CLEAN), (SELECT SUM(NUMBER_OF_ACCIDENTS) FROM A2_FACT_ACCIDENT)
FROM DUAL
```

The cleaned accident count is 993 because one source row has no accident identifier. The cleaned booking count is 10,000 because the exact duplicate T1218 is counted once. Maintenance row count remains 1,001; only the invalid amount is quarantined through its value and flag.

VERSION 1 TEST SUITE

C4. Required analytical queries (1-3)

Q1. IT faculty bus bookings in July

```
SELECT SUM(f.NUMBER_OF_BOOKINGS) BOOKING_COUNT
FROM A2_FACT_BOOKING f JOIN A2_DIM_MONTH m ON f.MONTH_ID=m.MONTH_ID
JOIN A2_DIM_BODY_TYPE c ON f.BODY_TYPE_ID=c.BODY_TYPE_ID
WHERE f.FACULTY_ID='FIT' AND m.MONTH_NUMBER=7 AND c.CAR_BODY_TYPE='BUS'
```

Verified result: 82 bookings

Q2. Bookings by age group

```
SELECT a.AGE_GROUP_NAME, NVL(SUM(f.NUMBER_OF_BOOKINGS),0) BOOKING_COUNT
FROM A2_DIM_AGE_GROUP a LEFT JOIN A2_FACT_BOOKING f ON a.AGE_GROUP_ID=f.AGE_GROUP_ID
GROUP BY a.AGE_GROUP_ID, a.AGE_GROUP_NAME ORDER BY a.AGE_GROUP_ID
```

Verified result: Young 4,085; Middle-aged 5,062; Old-aged 853; Other/unknown 0

Q3. Maintenance records associated with research centre CE04

```
SELECT SUM(f.NUMBER_OF_MAINT_RECORDS) MAINTENANCE_COUNT FROM A2_FACT_MAINT f
WHERE EXISTS (SELECT 1 FROM A2_BR_CENTRE_GROUP b
WHERE b.CENTRE_GROUP_ID=f.CENTRE_GROUP_ID AND b.CENTRE_ID='CE04')
```

Verified result: 627 records

For Q3, EXISTS is deliberate: it tests centre membership without joining the bridge into the aggregation and thereby avoids multiplying a maintenance group across all centres.

MINI BUS MAINTENANCE COST

C4. Required analytical query 4

Q4. Total known Mini Bus maintenance cost by maintenance type, with unknown amounts shown separately.

```
SELECT t.MAINTENANCE_TYPE,t.MAINTENANCE_DESCRIPTION,SUM(f.TOTAL_MAINTENANCE_COST) KNOWN_COST,
       SUM(f.NUMBER_OF_UNKNOWN_COSTS) UNKNOWN_COST_RECORDS
FROM A2_FACT_MAINT f JOIN A2_DIM_BODY_TYPE c ON f.BODY_TYPE_ID=c.BODY_TYPE_ID
JOIN A2_DIM_MAINT_TYPE t ON f.MAINTENANCE_TYPE=t.MAINTENANCE_TYPE
WHERE c.CAR_BODY_TYPE='MINI BUS'
GROUP BY t.MAINTENANCE_TYPE,t.MAINTENANCE_DESCRIPTION ORDER BY t.MAINTENANCE_TYPE
```

Type	Description	Known cost	Unknown amounts
M001	System Upgrade	\$7,300	0
M002	Hardware Upgrade:Sensor	\$14,200	1
M003	Hardware Upgrade:Lidar system	\$17,700	0
M004	Battery replacement	\$18,400	0
M005	Regular maintenance	\$31,000	0

The Mini Bus known-cost total is \$88,600. One M002 record has an invalid source amount and is therefore excluded from money totals but included in the unknown-cost count.

ACCIDENT SUBJECT TEST SUITE

C4. Required analytical queries (5-7)

Q5. Accidents for Car01 in ZoneA

```
SELECT SUM(f.NUMBER_OF_ACCIDENTS) ACCIDENT_COUNT FROM A2_FACT_ACCIDENT f
WHERE f.ACCIDENT_ZONE='ZONEA' AND EXISTS (SELECT 1 FROM A2_BR_CAR_GROUP b
WHERE b.CAR_GROUP_ID=f.CAR_GROUP_ID AND b.REGISTRATION_NO='CAR01')
```

Verified result: 10 accidents

Q6. ZoneB accidents for Car06 by error code

```
SELECT e.ERROR_CODE,NVL(SUM(f.NUMBER_OF_ACCIDENTS),0) ACCIDENT_COUNT
FROM A2_DIM_ERROR e LEFT JOIN A2_FACT_ACCIDENT f ON e.ERROR_CODE=f.ERROR_CODE
AND f.ACCIDENT_ZONE='ZONEB' AND EXISTS (SELECT 1 FROM A2_BR_CAR_GROUP b
WHERE b.CAR_GROUP_ID=f.CAR_GROUP_ID AND b.REGISTRATION_NO='CAR06')
GROUP BY e.ERROR_CODE ORDER BY e.ERROR_CODE
```

Verified result: Error001 2; Error002 0; Error003 3; Error004 1; Error005 1; UNKNOWN 0

Q7. Severe-damage accidents involving Car05

```
SELECT SUM(f.NUMBER_OF_ACCIDENTS) ACCIDENT_COUNT FROM A2_FACT_ACCIDENT f
WHERE f.DAMAGE_SEVERITY='SEVERE DAMAGE' AND EXISTS (SELECT 1 FROM A2_BR_CAR_GROUP b
WHERE b.CAR_GROUP_ID=f.CAR_GROUP_ID AND b.REGISTRATION_NO='CAR05')
```

Verified result: 7 accidents

All car questions use EXISTS against A2_BR_CAR_GROUP. This counts each accident once when the requested car is a group member and does not allocate or duplicate multi-car accidents.

BUSINESS ANALYSIS

C5. Power BI Web - error severity

A clustered bar chart in Power BI Web compares all recorded incidents with severe-damage incidents for each cleaned error category. Values were imported from the verified Oracle Version 1 query output.

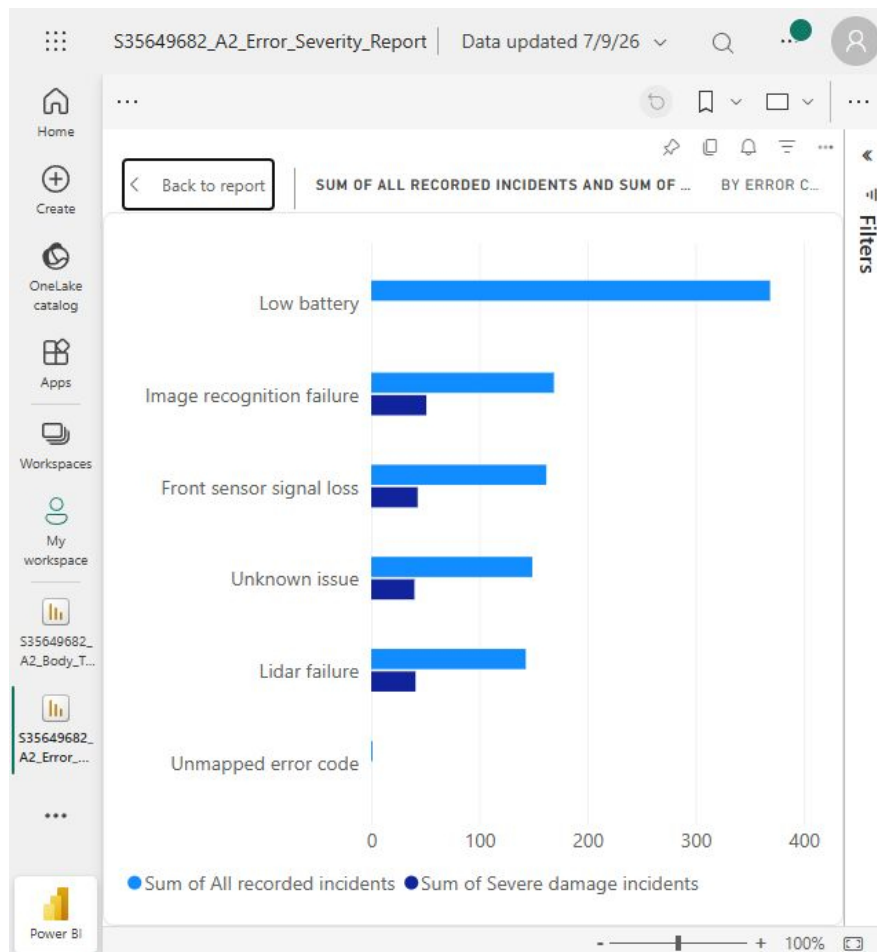


Figure 14. Power BI Web visual: all incidents versus severe-damage incidents by error category.

Error	All	Severe	Severe share
Error001	169	51	30.2%
Error002	369	0	0.0%
Error003	162	43	26.5%
Error004	143	41	28.7%
Error005	149	40	26.8%
UNKNOWN	1	0	0.0%
Total	993	175	17.6%

Reconciliation: all six categories total 993 valid accidents. UNKNOWN is the single unmapped Error010 record, not the 149 records classified as Error005 (Unknown issue).

Finding: Low battery is the most frequent recorded category (369) but has no severe records. Image-recognition failure has the largest severe count (51). Front-sensor, lidar and unknown-issue categories also show substantial severe counts, so they should not be ignored.

INVESTMENT EVIDENCE

C5. Power BI Web - body-type economics

The second Power BI Web report compares known maintenance cost per booking by body type. This normalises total valid maintenance expenditure by observed booking volume and gives a more decision-useful operating-cost indicator than total cost alone.

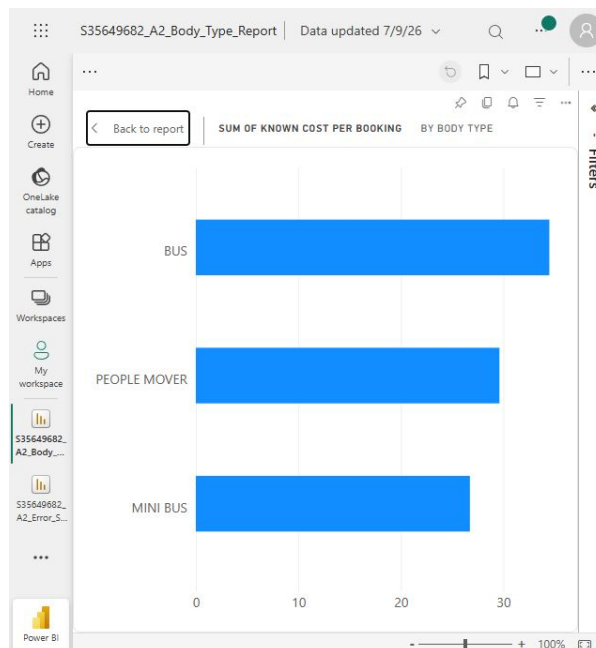


Figure 15. Power BI Web visual: known maintenance cost per booking by body type.

Body type	Bookings	Maint. records	Known cost	Unknown	Car involvements	Cost/booking
BUS	3,291	355	\$113,500	0	305	\$34.49
MINI BUS	3,313	312	\$88,600	1	324	\$26.74
PEOPLE MOVER	3,396	334	\$100,600	0	371	\$29.62

Finding: demand is tightly grouped across the three body types, but Mini Bus has both the lowest maintenance-record count and the lowest known cost per booking. People Mover has the highest bookings and the highest raw car-involvement count; this is an operational observation, not a causal accident-risk rate.

DECISION AND SAFEGUARDS

C5. Suggestion to MonCity

Suggestion (229 words; maximum 500)

MonCity should invest first in a controlled Mini Bus procurement pilot, subject to a short validation period. Mini Bus recorded 3,313 bookings, close to the highest-demand body type (People Mover, 3,396), but it produced the lowest known maintenance cost per booking at \$26.74. It also had the fewest maintenance records (312 versus 334 for People Mover and 355 for Bus). Together, these measures indicate the strongest balance of demand and observed maintenance burden. The pilot should track full maintenance cost, utilisation and passenger-capacity fit before fleet-wide expansion because one Mini Bus maintenance amount is unknown and the analysis does not include acquisition price, energy use or revenue.

Safety investment should prioritise the image-recognition system. Image-recognition failure has the largest severe-damage count (51), ahead of front-sensor signal loss (43), lidar failure (41) and unknown issue (40). Low battery is the most frequent error (369) but has zero severe-damage records in this dataset, so it warrants reliability monitoring rather than being the first severe-harm engineering priority.

The accident evidence must be interpreted cautiously. Accident records do not contain dates or exposure distance, and a car's presence in a multi-car accident does not establish causation. Consequently, the reported car-involvement counts should not be described as accident rates. MonCity should add accident date/time, kilometres travelled, operating hours and causal attribution to the source system, then reassess the recommendation using incidents per exposure unit.

Decision controls

Control	Action	Decision trigger
Cost completeness	Resolve the one unknown Mini Bus amount	Recalculate cost/booking before final purchase
Exposure-normalised safety	Capture dates, kilometres and hours	Use incidents per 10,000 km, not raw involvements
Engineering priority	Root-cause Error001 and test mitigations	Track severe incidents after remediation
Pilot governance	Compare Mini Bus pilot against People Mover baseline	Expand only if cost and safety advantages persist

The recommendation uses associations found in the warehouse; it does not claim that an error category or body type caused an accident.

AUDIT TRAIL

Assumptions, limitations and reproducibility

Area	Assumption / limitation	Response
Old-aged wording	Specification wording 'over 60' conflicts with an observed age of exactly 60	Use 60-120 so every valid adult booking is assigned; disclose boundary
Maintenance cost	Negative amount cannot be treated as a valid expense	Set amount NULL and keep a separate unknown-cost count
Month	Version 1 question asks for July without a year	Month-of-year dimension in V1; full date restored in V2
Multi-membership	Teams and accidents can have multiple members	Determinant group dimensions, bridges and weight-sum checks
Accident time	No date/time or distance exposure in accident source	Avoid rates, trends and causal claims
Power BI	Visuals use aggregated Oracle output	Retain SQL result tables and Oracle reconciliation in the report

Reproducible files

File	Purpose
A2_01_exploration.sql	Read-only source profiling
A2_02_verified_build.sql	Fresh Version 1 deployment and cleaning views
A2_03_verified_validation.sql	Seven questions, C5 analyses and integrity checks
A2_04_verified_export.sql	Machine-readable Oracle result export
output/data/verified_oracle_results.json	Verified results used in report tables
output/evidence/*.png	DBeaver and Power BI Web screenshots

Verification statement

All numerical findings in this report come from Oracle queries executed against the implemented Version 1 model. The Power BI visuals use those same verified aggregates. Screenshots document the live DBeaver and Power BI Web states. The report does not alter any MONCITY operational table.

Source basis

FIT5137 Assignment 2 Specification v4.0; FIT5137 A2 rubric; MonCity Oracle schema. Oracle deployment and analysis completed 7 September 2026; table-structure screenshots refreshed 15 September 2026.

Appendix A Complete cleaning SQL

These complete statements are taken from A2_02_verified_build.sql. They document a fresh deployment into the student schema; they do not update MONCITY. Create the passenger view before the booking view. Do not rerun CREATE statements against existing objects.

Passenger references and booking deduplication

```
CREATE VIEW A2_V_PASSENGER_CLEAN AS
SELECT p.PASSENGERID, p.PASSENGERAGE,
       NVL(f.FACULTYID, 'UNKNOWN') AS FACULTY_ID
FROM MONCITY.PASSENGER p LEFT JOIN MONCITY.FACULTY f
  ON UPPER(TRIM(p.FACULTYID))=UPPER(TRIM(f.FACULTYID));

CREATE VIEW A2_V_BOOKING_CLEAN AS
SELECT DISTINCT b.BOOKINGID AS BOOKING_ID, b.BOOKINGDATE AS BOOKING_DATE,
  EXTRACT(MONTH FROM b.BOOKINGDATE) AS MONTH_ID,
  NVL(p.FACULTY_ID, 'UNKNOWN') AS FACULTY_ID,
  CASE WHEN p.PASSENGERAGE BETWEEN 18 AND 35 THEN 1
        WHEN p.PASSENGERAGE BETWEEN 36 AND 59 THEN 2
        WHEN p.PASSENGERAGE BETWEEN 60 AND 120 THEN 3 ELSE 0 END AS AGE_GROUP_ID,
  UPPER(TRIM(b.REGISTRATIONNO)) AS REGISTRATION_NO
FROM MONCITY.BOOKING b LEFT JOIN A2_V_PASSENGER_CLEAN p
  ON b.PASSENGERID=p.PASSENGERID
WHERE b.BOOKINGID IS NOT NULL;
```

Maintenance amount and unknown cost flag

```
CREATE VIEW A2_V_MAINT_CLEAN AS
SELECT MAINTENANCEID AS MAINTENANCE_ID,
  MAINTENANCEDATE AS MAINTENANCE_DATE,
  UPPER(TRIM(REGISTRATIONNO)) AS REGISTRATION_NO,
  MAINTENANCETYPE AS MAINTENANCE_TYPE, TEAMID AS TEAM_ID,
  CASE WHEN MAINTENANCECOST >= 0 THEN MAINTENANCECOST
        END AS MAINTENANCE_COST,
  CASE WHEN MAINTENANCECOST < 0 OR MAINTENANCECOST IS NULL
        THEN 1 ELSE 0 END AS COST_UNKNOWN
FROM MONCITY.MAINTENANCE;
```

Accident references and rejected source records

```
CREATE VIEW A2_V_ACCIDENT_CLEAN AS
SELECT a.ACCIDENTID AS ACCIDENT_ID,
  UPPER(TRIM(a.ACCIDENTZONE)) AS ACCIDENT_ZONE,
  NVL(e.ERRORCODE, 'UNKNOWN') AS ERROR_CODE,
  UPPER(TRIM(a.CAR_DAMAGE_SEVERITY)) AS DAMAGE_SEVERITY
FROM MONCITY.ACCIDENTINFO a LEFT JOIN MONCITY.ERROR e
  ON a.ERRORCODE=e.ERRORCODE
WHERE a.ACCIDENTID IS NOT NULL;

CREATE VIEW A2_V_REJECT_ACCIDENT AS
SELECT * FROM MONCITY.ACCIDENTINFO WHERE ACCIDENTID IS NULL;
```

Appendix A Dimension cleaning and unknown members

These dimension statements preserve unmapped references and standardise the two error-message spellings. UNKNOWN is separate from Error005, whose description is Unknown issue. The final view retains accidents with no car link.

Faculty unknown member

```
CREATE TABLE A2_DIM_FACULTY AS
SELECT FACULTYID FACULTY_ID,FACULTYNAME FACULTY_NAME,ZONE FACULTY_ZONE
FROM MONCITY.FACULTY
UNION ALL SELECT 'UNKNOWN','Unknown faculty','UNKNOWN' FROM DUAL;

ALTER TABLE A2_DIM_FACULTY ADD CONSTRAINT A2_PK_DIM_FACULTY
PRIMARY KEY (FACULTY_ID);
```

Corrected error descriptions and unmapped error member

```
CREATE TABLE A2_DIM_ERROR AS
SELECT ERRORCODE ERROR_CODE,
  REPLACE(REPLACE(ERRORMESSAGE,'systeem','system'),
    'Unknow issue','Unknown issue') ERROR_MESSAGE
FROM MONCITY.ERROR
UNION ALL SELECT 'UNKNOWN','Unmapped source error code' FROM DUAL;

ALTER TABLE A2_DIM_ERROR ADD CONSTRAINT A2_PK_DIM_ERROR
PRIMARY KEY (ERROR_CODE);
```

Disclosed age boundary and fallback member

```
CREATE TABLE A2_DIM_AGE_GROUP AS
SELECT 0 AGE_GROUP_ID,'Other / unknown' AGE_GROUP_NAME,
  CAST(NULL AS NUMBER) MIN_AGE,CAST(NULL AS NUMBER) MAX_AGE FROM DUAL
UNION ALL SELECT 1,'Young adult',18,35 FROM DUAL
UNION ALL SELECT 2,'Middle-aged adult',36,59 FROM DUAL
UNION ALL SELECT 3,'Old-aged adult',60,120 FROM DUAL;

ALTER TABLE A2_DIM_AGE_GROUP ADD CONSTRAINT A2_PK_DIM_AGE_GROUP
PRIMARY KEY (AGE_GROUP_ID);
```

Unknown car and accident membership fallback

```
CREATE TABLE A2_DIM_CAR AS
SELECT UPPER(TRIM(REGISTRATIONNO)) REGISTRATION_NO,CARMODEL CAR_MODEL,
  MANUFACTURINGYEAR MANUFACTURING_YEAR,
  UPPER(TRIM(CARBODYTYPE)) CAR_BODY_TYPE,NUMSEATS NUMBER_OF_SEATS
FROM MONCITY.CAR
UNION ALL SELECT 'UNKNOWN','Unknown car',NULL,'UNKNOWN',NULL FROM DUAL;

ALTER TABLE A2_DIM_CAR ADD CONSTRAINT A2_PK_DIM_CAR
PRIMARY KEY (REGISTRATION_NO);

CREATE VIEW A2_V_ACC_CAR AS
SELECT a.ACCIDENT_ID,
  NVL(UPPER(TRIM(c.REGISTRATIONNO)),'UNKNOWN') REGISTRATION_NO
FROM A2_V_ACCIDENT_CLEAN a LEFT JOIN MONCITY.CARACCIDENT c
  ON a.ACCIDENT_ID=c.ACCIDENTID;
```

Source: A2_02_verified_build.sql. SQL token checks confirm that the statements above match the verified deployment script; only whitespace and line breaks have been reformatted.